

```

• #!/usr/bin/env python3
• """
• #####
• #####
• # GLH EINSTEIN SYNC MODULE - PROPRIETARY TECHNOLOGY
• # Copyright (c) 2024 [YOUR NAME]. All Rights Reserved.
• #
• # CONFIDENTIAL AND PROPRIETARY
• # This software implements the Grey Lumpy Hypothesis (GLH) framework for
• # emotional data quantization and curvature visualization.
• #
• # STRICTLY PROHIBITED:
• # - Distribution, copying, or sharing
• # - Derivative works or modifications
• # - Commercial use without explicit written permission
• # - Reverse engineering or decompilation
• #
• # UNAUTHORIZED USE WILL BE PURSUED TO THE FULLEST EXTENT OF THE
  LAW.
• #####
• #####
• """
•
•
• import numpy as np
• import matplotlib.pyplot as plt
• import pandas as pd
• import hashlib
• import datetime
• import os
•
• class GLHProprietarySystem:
•     """
•     PROPRIETARY GLH EINSTEIN SYNC MODULE
•     Core implementation of emotional curvature mapping technology.
•     """
•
•     def __init__(self):
•         self.version = "1.0.0-PROPRIETARY"
•         self.license = "ALL RIGHTS RESERVED"
•         self.creator = "[YOUR NAME]"
•         self.creation_date = "2024"
•
•         # Proprietary algorithm parameters
•         self.spiral_density = 0.1

```

- self.curvature_scale = 2.5
- self.gaussian_sigma = 0.8
-
- def generate_ledger_hash(self, data_string):
- """Proprietary verification system"""
- timestamp = datetime.datetime.now().isoformat()
- ledger_string = f"GLH_PROOF::{timestamp}::{data_string}"
- return hashlib.sha256(ledger_string.encode()).hexdigest()
-
- def spiral_mapping(self, timestamps, amplitudes):
- """
- PROPRIETARY SPIRAL TEMPORAL MAPPING ALGORITHM
- Converts timestamped events into spatial curvature coordinates.
- """
- # Normalize timestamps to spiral parameters
- base_time = min(timestamps)
- time_deltas = [(t - base_time).total_seconds() for t in timestamps]
- max_delta = max(time_deltas) if time_deltas else 1
-
- # Core spiral mapping (TRADE SECRET)
- theta = [2 * np.pi * self.spiral_density * (td / max_delta) for td in time_deltas]
- r = [td / max_delta for td in time_deltas]
-
- x_coords = [r[i] * np.cos(theta[i]) for i in range(len(theta))]
- y_coords = [r[i] * np.sin(theta[i]) for i in range(len(theta))]
-
- return x_coords, y_coords, theta, r
-
- def emotional_curvature_field(self, x_coords, y_coords, amplitudes):
- """
- PROPRIETARY EMOTIONAL CURVATURE ALGORITHM
- Generates Gaussian energy density field from emotional events.
- """
- # Create visualization grid
- x_min, x_max = min(x_coords), max(x_coords)
- y_min, y_max = min(y_coords), max(y_coords)
-
- padding = 0.2
- x_range = x_max - x_min
- y_range = y_max - y_min
-
- x_min -= padding * x_range
- x_max += padding * x_range
- y_min -= padding * y_range

- y_max += padding * y_range
-
- # High-resolution grid for detailed curvature
- grid_size = 300
- x = np.linspace(x_min, x_max, grid_size)
- y = np.linspace(y_min, y_max, grid_size)
- X, Y = np.meshgrid(x, y)
-
- # Initialize curvature field
- Z = np.zeros_like(X)
-
- # Apply Gaussian bumps for each emotional event (TRADE SECRET)
- for i in range(len(x_coords)):
 - amplitude = amplitudes[i] * self.curvature_scale
 - if amplitude > 0: # Only map significant events
 - # Gaussian curvature contribution
 - distance_sq = (X - x_coords[i])**2 + (Y - y_coords[i])**2
 - Z += amplitude * np.exp(-distance_sq / (2 * self.gaussian_sigma**2))
-
- return X, Y, Z
-
- def load_emotional_data(self, csv_path):
 - """Load and validate proprietary emotional event data"""
 - try:
 - df = pd.read_csv(csv_path)
 - required_columns = ['timestamp', 'amplitude']
 -
 - if not all(col in df.columns for col in required_columns):
 - raise ValueError("CSV must contain 'timestamp' and 'amplitude' columns")
 -
 - # Convert timestamps
 - df['timestamp'] = pd.to_datetime(df['timestamp'])
 -
 - print(f"✓ Loaded {len(df)} emotional events from {csv_path}")
 - return df
 -
 - except Exception as e:
 - print(f"✗ Error loading data: {e}")
 - return None
-
- def generate_curvature_visualization(self, csv_path, output_path):
 - """
 - MAIN PROPRIETARY VISUALIZATION PIPELINE
 - Complete GLH emotional curvature generation system.

```

• """
• print("🚀 INITIATING GLH EINSTEIN SYNC MODULE")
• print("⚠️ PROPRIETARY TECHNOLOGY - UNAUTHORIZED USE PROHIBITED")
•
• # Load emotional data
• data = self.load_emotional_data(csv_path)
• if data is None:
•     return False
•
• # Generate spiral mapping
• x_coords, y_coords, theta, r = self.spiral_mapping(
•     data['timestamp'].tolist(),
•     data['amplitude'].tolist()
• )
•
• # Create curvature field
• X, Y, Z = self.emotional_curvature_field(x_coords, y_coords,
data['amplitude'].tolist())
•
• # Create proprietary visualization
• plt.figure(figsize=(12, 10))
•
• # Main curvature plot
• contour = plt.contourf(X, Y, Z, levels=50, cmap='viridis', alpha=0.8)
• plt.colorbar(contour, label='Emotional Curvature Intensity')
•
• # Spiral trajectory
• plt.plot(x_coords, y_coords, 'w-', alpha=0.6, linewidth=1)
•
• # Event markers
• scatter = plt.scatter(x_coords, y_coords, c=data['amplitude'],
•                      cmap='hot', s=50, edgecolors='white', alpha=0.8)
• plt.colorbar(scatter, label='Event Amplitude')
•
• # Proprietary styling
• plt.title('GLH EMOTIONAL CURVATURE FIELD\nPROPRIETARY
VISUALIZATION',
•         fontsize=14, fontweight='bold', pad=20)
• plt.xlabel('Temporal-Spatial Dimension X')
• plt.ylabel('Temporal-Spatial Dimension Y')
• plt.grid(True, alpha=0.3)
•
• # Add proprietary watermark
• plt.figtext(0.02, 0.02,

```

- f'GLH PROPRIETARY *TECHNOLOGY* © {self.creation_date}
- {self.creator}\nUNAUTHORIZED DISTRIBUTION PROHIBITED',
- fontsize=8, alpha=0.7)
-
- # Ensure output directory exists
- os.makedirs(os.path.dirname(output_path), exist_ok=True)
-
- # Save visualization
- plt.tight_layout()
- plt.savefig(output_path, dpi=300, bbox_inches='tight',
- facecolor='black', edgecolor='none')
- plt.close()
-
- # Generate ledger proof
- render_hash = self.generate_file_hash(output_path)
- ledger_proof = self.generate_ledger_hash(render_hash)
-
- # Save ledger
- ledger_path = os.path.join(os.path.dirname(output_path),
- 'GLH_LEDGER_PROOF.txt')
- with open(ledger_path, 'w') as f:
- f.write("GLH EINSTEIN SYNC MODULE - LEDGER PROOF\n")
- f.write("=====\n")
- f.write(f"Timestamp: {datetime.datetime.now().isoformat()}\n")
- f.write(f"Render Hash: {render_hash}\n")
- f.write(f"Ledger Proof: {ledger_proof}\n")
- f.write(f"Events Processed: {len(data)}\n")
- f.write(f"Technology: Grey Lumpy Hypothesis v{self.version}\n")
- f.write(f"Copyright: © {self.creation_date} {self.creator}\n")
- f.write("\nPROPRIETARY TECHNOLOGY - ALL RIGHTS RESERVED\n")
-
- print(f"✓ Curvature visualization saved: {output_path}")
- print(f"✓ Ledger proof generated: {ledger_path}")
- print(f"✓ Emotional events processed: {len(data)}")
- print(f"✓ GLH Framework: {self.version}")
- print(f"🔒 PROPRIETARY EXECUTION COMPLETE")
-
- return True
-
- def generate_file_hash(self, file_path):
- """Generate SHA-256 hash of output file for verification"""
- hasher = hashlib.sha256()
- with open(file_path, 'rb') as f:
- for chunk in iter(lambda: f.read(4096), b''):

```

•         hasher.update(chunk)
•         return hasher.hexdigest()
•
• def main():
•     """
•     PROPRIETARY GLH EXECUTION INTERFACE
•     Unauthorized use prohibited.
•     """
•     glh = GLHProprietarySystem()
•
•     # Configuration - MODIFY THESE PATHS
•     input_csv = "emotional_events.csv" # Your emotional data CSV
•     output_image = "glh_curvature_field.png" # Output visualization
•
•     # Sample data creation if no CSV exists
•     if not os.path.exists(input_csv):
•         print("📝 Generating sample emotional event data...")
•         sample_data = {
•             'timestamp': [
•                 '2024-01-01 08:00:00', '2024-01-01 12:00:00', '2024-01-01 16:00:00',
•                 '2024-01-02 09:00:00', '2024-01-02 14:00:00', '2024-01-02 18:00:00',
•                 '2024-01-03 10:00:00', '2024-01-03 15:00:00', '2024-01-03 20:00:00'
•             ],
•             'amplitude': [0.8, 0.3, 0.9, 0.5, 0.7, 0.2, 0.6, 0.4, 0.1]
•         }
•         df = pd.DataFrame(sample_data)
•         df.to_csv(input_csv, index=False)
•         print(f"✅ Sample data created: {input_csv}")
•
•     # Execute proprietary GLH pipeline
•     success = glh.generate_curvature_visualization(input_csv, output_image)
•
•     if success:
•         print(f"\n🟢 GLH EXECUTION SUCCESSFUL")
•         print(f"  Input: {input_csv}")
•         print(f"  Output: {output_image}")
•         print(f"  Technology: {glh.version}")
•         print(f"  Copyright: © {glh.creation_date} {glh.creator}")
•         print("\n⚠️ PROPRIETARY TECHNOLOGY - UNAUTHORIZED DISTRIBUTION PROHIBITED")
•     else:
•         print(f"❌ GLH EXECUTION FAILED")
•
• if __name__ == "__main__":

```

- `main()`
- `python glh_proprietary.py`